

Università degli Studi di Salerno

Corso di Ingegneria del Software

Tasky Test Plan Versione 0.6



Data: 19/12/2025

Progetto: Tasky	Versione: 0.6
Documento: Test Plan	Data: 19/12/2025

Coordinatore del progetto:

Nome	Matricola
Luca Vincenzo Lambiase Fontana	0512113338

Partecipanti:

Nome	Matricola
Ilaria Lastra	0512120280
Rinaldo Perna	0512119593

Scritto da:	Ilaria Lastra
--------------------	---------------

Revision History

Data	Versione	Descrizione	Autore
01/10/2025	0.1	Presentazione Proposta di Progetto	Lastra Ilaria
10/10/2025	0.2	Presentazione Problem Statement con descrizione degli scenari principali dell'applicazione, i suoi requisiti e gli ambienti target	Lastra Ilaria
20/10/2025	0.3	Presentazione del Requirements Analysis Document, con perfezionamento dei requisiti esposti nel Problem Statement e definizione dei criteri di successo e dello scopo e ambito del sistema	Lastra Ilaria
08/11/2025	0.3.1	Presentazione dell'Object Model e del Dynamic Model, con Class Diagram, Sequence Diagram e Statechart Diagram relativi al Requirements Analysis Document	Lastra Ilaria
19/11/2025	0.4	Presentazione del System Design Document, con discussione dei design goals, dell'architettura e mappatura hardware, software.	Lastra Ilaria, Luca Vincenzo Lambiase Fontana, Rinaldo Perna
19/12/2025	0.5	Presentazione dell'Object Design Document con discussione dei compromessi della progettazione ad oggetti, linee guida per la documentazione dell'interfaccia e packaging dei sottosistemi	Ilaria Lastra, Luca Vincenzo Lambiase Fontana, Rinaldo Perna

Progetto: Tasky	Versione: 0.6
Documento: Test Plan	Data: 19/12/2025

19/12/2025	0.6	Presentazione del Test Plan Document	Ilaria Lastra, Luca Vincenzo Lambiase Fontana, Rinaldo Perna
------------	-----	--------------------------------------	--

Sommario

<i>1.Introduzione</i>	<i>6</i>
<i>2.Relationship to other documents</i>	<i>6</i>
<i>3.System overview</i>	<i>6</i>
<i>4.Features to be tested/not to be tested</i>	<i>6</i>
<i>5.Pass/Fail criteria</i>	<i>7</i>
<i>6.Approach</i>	<i>7</i>
<i>7.Suspension and resumption.....</i>	<i>7</i>
<i>8.Testing materials (hardware/software requirements).....</i>	<i>8</i>
<i>9.Test cases</i>	<i>8</i>
<i>10.Test schedule</i>	<i>9</i>

1.Introduzione

L'obiettivo di questo piano di test è definire il framework operativo per la verifica e la validazione del sistema Tasky. Il documento mira a coordinare le attività dei nostri tester per assicurare che il software di gestione task e progetti funzioni correttamente entro i tempi stabiliti, minimizzando i costi legati ai bug post-rilascio. Il focus principale è sulla correttezza della logica di business (backend Flask) e sulla robustezza dell'interfaccia utente.

2.Relationship to other documents

Il Test Plan è strettamente legato ai seguenti documenti di progetto:

- **RAD (Requirements Analysis Document):** Ogni caso di test è tracciato verso i requisiti funzionali (es. *Gestione Task, Autenticazione*) e non funzionali (es. *Sicurezza, Performance*).
- **SDD (System Design Document):** I test di integrazione seguono la scomposizione in sottosistemi (Three-Tier Architecture) definita nel SDD.
- **Schema di denominazione:** Per garantire la tracciabilità, utilizzeremo il prefisso **TP-[ID]** (Test Plan) mappato sui casi d'uso **UC-[ID]** del RAD.

3.System overview

Il testing unitario si concentrerà sui singoli componenti atomici del sistema, identificati nel SDD nella sezione di Subsystem Decomposition. La progettazione dei test di sistema inoltre seguirà la tecnica della “**Category Partition**”, dove andremo a scomporre i domini di input in partizioni di equivalenza per massimizzare la copertura degli errori.

- **Granularità:** Il test di unità verrà eseguito a livello di singoli metodi delle classi Service e controller API
- **Componenti Chiave:**
 - **User Management:** Servizio di autenticazione
 - **Department Management:** Logica di gestione dei membri del dipartimento
 - **Project management:** CRUD dei progetti
 - **Task Management:** Unità minima di lavoro, gestione degli stati e delle scadenze.
- **Dipendenze:** Le dipendenze che riguardano i seguenti moduli sono: il modulo Task dipende strettamente dal modulo Project, che a sua volta dipende da Department.

4.Features to be tested/not to be tested

1. Caratteristiche da testare:

- **Autenticazione:** Verifica del Bearer Token e del Role-Based Access Control (RBAC).
- **Ciclo di vita del Task:** Transizioni tra gli stati: *In corso, Completato* e *Sospeso*.
- **Integrità Referenziale:** Cancellazione di un progetto e impatto sui task associati.
- **Corretta visualizzazione dei dati richiesti:** in riferimento alle liste di progetti o membri di uno specifico dipartimento
- **Gestione Scadenze:** Notifica o cambiamento di stato automatico alla violazione della data

fine.

2. Caratteristiche da non testare:

- **Librerie di terze parti:** Non testeremo le funzionalità interne del framework Flask o SQLAlchemy.
- **Connessione Internet:** Si assume una connettività stabile per i test funzionali, a meno di specifici test di resilienza.

5.Pass/Fail criteria

I criteri di superamento/fallimento che dovranno essere presi in considerazione per lo svolgimento dei test richiesti sono:

- ☐ **Pass (Superato):** Il sistema produce l'output atteso definito nel RAD senza errori di runtime.
- ☐ **Fail (Fallito):** Ogni deviazione dal comportamento atteso, inclusi errori di validazione dati, crash del server o violazioni della sicurezza (accesso non autorizzato).
- ☐ **Criterio Finale:** Il sistema è considerato pronto per il rilascio solo se la maggior parte (100%) dei test critici e buona parte (90%) dei test non critici sono "Superati".

6.Approach

L'approccio scelto per la fase di test sarà di natura ibrida:

- Per quanto riguarda il test di unità, dunque quello che toccherà le varie componenti del nostro sistema singolarmente, avremo un approccio White-Box. Avremo quindi una verifica dei singoli metodi nel repository.py e utils.py per garantire che le query SQL e i decoratori di sicurezza funzionino come previsto
- Per il test di integrazione invece avremo un approccio Black Box, con utilizzo di script automatizzati (curl_test.sh) per verificare l'interazione tra i moduli tramite chiamate http agli endpoint REST.
- Per il System Testing infine avremo un approccio volto alla GUI, con utilizzo dell'interfaccia web/tester per simulare flussi utente completi (es creazione progetto, aggiunta task, verifica visibilità dipendente). La progettazione e la specifica di tali casi di test segue la tecnica **Category Partition(CP)**. Per ogni unità funzionale di sistema (ES. Login, Gestione Task, Progetti) sono stati individuati i parametri di input (**categorie**) e le relative classi di equivalenza (**scelte**), permettendo di derivare sistematicamente sia i casi di test nominali che quelli di gestione dell'errore. Questo garantisce una copertura ottimale dei requisiti funzionali richiesti e descritti nel RAD.

7.Suspension and resumption

La fase di test può essere soggetta a sospensione o ripresa sulla base di criteri specifici definiti nella seguente sezione:

- ☐ **Sospensione:** Il testing verrà sospeso se oltre il 15% dei test case fallisce o se un difetto "bloccante" impedisce l'accesso al sistema (es. fallimento totale del modulo di Login).
- ☐ **Ripresa:** I test riprenderanno solo dopo la correzione documentata del bug bloccante. Alla ripresa, verrà eseguito un intero ciclo di **Regression Testing** per garantire che la correzione non

abbia introdotto nuovi difetti.

8. *Testing materials (hardware/software requirements)*

Per l'esecuzione del test sono necessari:

- Software Server: Python 3.x, Flask, Database (MySQL), dipendenze listate in requirements.txt.
- Tools di test: Bash (per curl_test.sh), Browser Web e Postman (opzionale)
- Ambiente Mobile: Android Studio / Emulatore Android (per i test di integrazione notifiche Push).
- Dati di Test: Script seed_test_data.py per popolare il DB con utenti e dipartimenti fittizi."

9. *Test cases*

I principali casi di test selezionati per il nostro Test Plan sono:

- **TC_01 (Login):** Verrà effettuata la verifica dell'accesso con le credenziali valide e verrà controllata la corretta generazione del token per accesso velocizzato alle future sessioni. Il seguente caso di test è associabile allo **UC1: Login** del nostro RAD
- **TC_02 (Aggiunta Task):** Relativa verifica del corretto inserimento della task definita tramite relativo form, con data di scadenza valida. Il seguente caso di test è associabile allo **UC2: Aggiunta di un task** del nostro RAD
- **TC_03 (Sospensione Task):** Verifica che solo un utente con ruolo di "Manager" possa sospendere un task, allegando la motivazione della sua scelta nei confronti del task, del quale verificare la corretta selezione. Il seguente caso di test è associabile allo **UC5: Sospensione di un task** del nostro RAD
- **TC_04 (Eliminazione Task):** Verifica che solo un utente con il ruolo di "Manager" possa eliminare un task, verificando la sua corretta selezione e relativa eliminazione anche all'interno dell'archivio persistente dei dati. Il seguente caso di test è associabile allo **UC4: Eliminazione di un task** del nostro RAD
- **TC_05 (Visualizzazione Progetti):** Verifica del filtraggio dei progetti per dipartimento di appartenenza del manager. Il seguente caso di test è associabile allo **UC12: Visualizzazione lista dei progetti** del nostro RAD
- **TC_06 (Aggiunta Progetto):** Verifica del corretto inserimento di un progetto definita tramite relativo form. Il seguente caso di test è associabile allo **UC6: Aggiunta progetto** del nostro RAD
- **TC_07 (Eliminazione Progetto):** Verifica della corretta eliminazione del progetto selezionato da parte di un manager, con relativo controllo del cascading della seguente operazione anche sui task facenti parte del progetto eliminato. Il seguente caso di test è associabile allo **UC7: Eliminazione progetto** del nostro RAD
- **TC_08 (Scheduler Scadenze):** Verifica che l'invocazione del job automatico porti i task con data_fine passata allo stato 'Sospeso'. Il seguente caso di test è associabile allo **UC8: Scheduler scadenze automatico** del nostro RAD
- **TC_09 (Visibilità Dipartimento):** Verifica che un Dipendente possa vedere la lista dei colleghi del proprio dipartimento (HTTP 200) ma non di altri (HTTP 403). Il seguente caso di test è associabile allo **UC9: Visualizzazione colleghi del proprio dipartimento** del nostro RAD.
- **TC_10 (Progetti per Manager):** Verifica che, dato un Manager, l'API restituisca tutti i

progetti in cui egli ha assegnato almeno un task. Il seguente caso di test è associabile allo **UC10: Progetti per Manager** del nostro RAD.

I nostri casi di test seguono la tecnica della “**Category Partition**” come illustrato nel documento di **Test Cases Specification**; dunque, ogni singolo test case indicato nel seguente documento sarà allegato a test cases singolari appartenenti alle categorie descritte per toccare tutte le possibili combinazioni fra le scelte legate alle categorie prese in esame per ogni singola operazione e metodo

10. Test schedule

- **Responsabilità:** Il Team Lead supervisiona il piano; gli sviluppatori eseguono Unit Test, Integration e System Test.
- **Rischi:** Ritardi nell'implementazione dell'API RESTful potrebbero slittare i test di integrazione frontend.
- **Pianificazione:**
 - Settimana 1-2: Unit Testing (moduli core).
 - Settimana 3: Integration Testing (API/DB).
 - Settimana 4: System Testing e Regression Testing finale.